HUMSAFAR

A Bi-Directional Remote Intervention and Care Coordination System for Vulnerable Individuals

Software Design Specification (SDS)

Version 1.0 — Final Locked Baseline

Derived from Humsafar SRS v1.0 (Final Locked Baseline)

Document Status: FINAL & LOCKED

Department: Computer Science (BSSE)

Team: Muhammad Abdullah (Team Lead), Muhammad Talha Khan, Syed Sarim Jawad Zaidi

Submitting to: S.M. Khalid Jamal

Revision History

Version	Date	Description	Author
0.1	Design Phase	Initial draft derived from SRS v1.0	Project Team
1.0	Design Phase	Final locked baseline; complete architectural, database, API, and interface design	Project Team

Table of Contents

4. Introduction **5**

4.1 Purpose 5

4.2 Scope 5

4.3 Definitions, Acronyms, and Abbreviations 5

4.4 References 6

4.5 Document Overview 6

5. System Overview **7**

6. Architectural Design **8**

6.1 Overall Architecture 8

6.2 Architectural Style 8

6.3 Design Principles 9

7. Module Design **10**

7.1 Module Responsibilities 10

7.2 Component Relationships 10

8. Component Design **12**

9. Database Design **14**

9.1 Entity-Relationship Diagram 14

9.2 Database Schema 14

9.3 Relationships 16

9.4 Constraints 16

10. User Interface Design **17**

10.1 Screen Descriptions 17

10.2 Navigation Flow 18

11. API and Backend Design **19**

11.1 Endpoints 19

11.2 Request/Response Structures 20

11.3 Authentication 20

12. Sequence Diagrams **21**

13. Activity Diagrams **24**

14. Class Diagram **26**

15. State Diagrams **27**

16. Deployment Architecture **29**

17. Security Design **30**

18. Error Handling and Exception Strategy **31**

19. Logging and Monitoring Strategy **32**

20. Performance and Scalability Considerations **33**

21. Design Constraints **34**

22. Design Decisions and Rationale **35**

23. Requirements Traceability Matrix **36**

24. Appendices **38**

24.1 Technology Stack Summary 38

24.2 Glossary 38

4. Introduction

4.1 Purpose

This Software Design Specification (SDS) defines the architectural, modular, database, interface, and component-level design of Humsafar. It translates the approved Software Requirements Specification (SRS v1.0, Final Locked Baseline) into a concrete design suitable for implementation, and serves as the primary reference for development, database construction, API implementation, and testing.

4.2 Scope

This document covers the complete design of the Humsafar system as scoped in SRS Section 1.4: identity and access management, Guardian-Dependent relationship management, location and geofencing, layered emergency detection and escalation, communication, medication and medical records management, wearable health-band integration, healthcare directory and home-care marketplace, camera-based surveillance, security and data management, accessibility, and analytics. Items explicitly out of scope in the SRS (auto-answer emergency audio, continuous CCTV recording, in-app payments and provider vetting, voice-guided interaction, custom wearable hardware, iOS support) remain out of scope for this design.

4.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification
SDS	Software Design Specification
MVVM	Model-View-ViewModel — the presentation-layer architectural pattern used in the Android applications
REST	Representational State Transfer — the architectural style used for the backend API
JWT	JSON Web Token — the token format used for stateless API authentication
ORM	Object-Relational Mapping — the data-access abstraction layer over the relational database
BLE / GATT	Bluetooth Low Energy / Generic Attribute Profile — the wireless protocol and data format used for wearable band integration
RBAC	Role-Based Access Control
TLS	Transport Layer Security
ERD	Entity-Relationship Diagram
UML	Unified Modeling Language
FCM	Firebase Cloud Messaging — the push notification delivery service

RTC	Real-Time Communication (voice/video)
Guardian, Dependent, Primary/Secondary Guardian, Dependent Mode, KYC, Geofence, Escalation Chain	As defined in SRS Section 7.1 (Glossary), retained without modification in this document

4.4 References

- Humsafar Software Requirements Specification, Version 1.0 (Final Locked Baseline)
- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications
- IEEE Std 1016-2009, IEEE Standard for Information Technology — Systems Design — Software Design Descriptions

4.5 Document Overview

Section 5 provides a system overview. Section 6 defines the overall architecture. Sections 7 and 8 define module and component design. Section 9 defines the database design. Sections 10 and 11 define the user interface and API design. Sections 12 through 15 present the behavioral models (sequence, activity, class, and state diagrams). Section 16 defines the deployment architecture. Sections 17 through 20 address security, error handling, logging, and performance. Sections 21 and 22 record design constraints and rationale. Section 23 provides the Requirements Traceability Matrix, and Section 24 contains supporting appendices.

5. System Overview

Humsafar is structured as a three-tier, client-server system. Two native Android applications — a Guardian application and a Dependent application — communicate with a backend server over a RESTful API secured with JWT-based authentication. The backend is organized into eight cohesive service modules, each corresponding to a domain group defined in the SRS (Section 4, Parts A through G, and the notification cross-cutting concern). The system integrates with five categories of external service: a wearable health band (via Bluetooth Low Energy), a KYC/liveness identity verification provider, a push notification service, a real-time communication SDK for voice and video, and an SMS/voice gateway for the external emergency contact fallback.

The system's six actors — Primary Guardian, Secondary Guardian, Dependent, External Emergency Contact, Home-Care Professional, and the Backend System itself — interact through 22 use cases as defined in SRS Section 6. This design specification maps each of those use cases, and the 47 functional requirement groups behind them, to concrete modules, components, database entities, and API endpoints.

6. Architectural Design

6.1 Overall Architecture

Humsafar follows a layered client-server architecture comprising a presentation layer (the two Android applications), an interface layer (a REST API gateway handling authentication and request routing), an application layer (eight domain-aligned service modules), and a data layer (a relational database, an in-memory cache/queue, and object storage for documents and media).

Figure 6.1 — Overall System Architecture (Layered Client-Server)

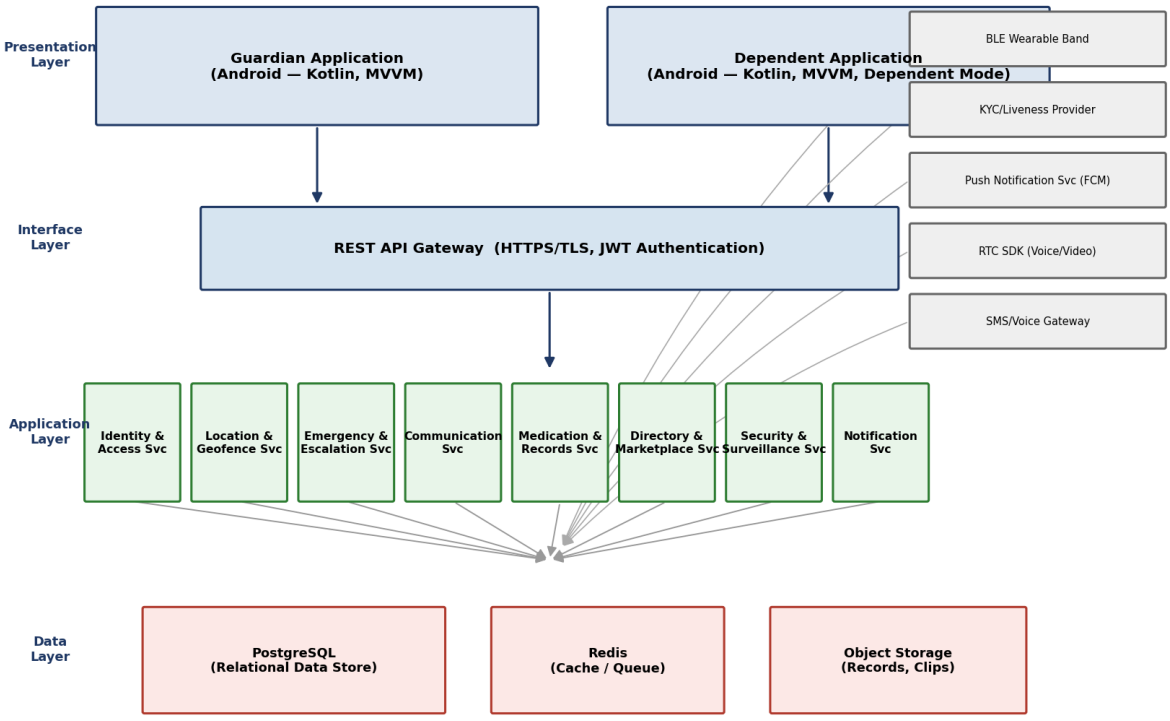


Figure 6.1 — Overall System Architecture (Layered Client-Server)

6.2 Architectural Style

The system adopts a **layered, modular-monolith architecture** exposed through a RESTful API, rather than a microservices architecture. This choice is deliberate: microservices introduce operational overhead (independent deployment pipelines, inter-service network reliability, distributed transaction management) that is disproportionate to a three-person development team operating within a single academic-year timeline (SRS Section 2.5). A modular monolith preserves clean separation between the eight service modules at the code level while deploying as a single unit, and can be decomposed into microservices later if the system's scale warrants it.

Communication between the mobile applications and the backend follows the REST architectural style over HTTPS, consistent with SRS Section 3.3/3.4. Asynchronous, event-driven communication (via the Redis queue)

is used internally for notification delivery and escalation timers, decoupling time-sensitive background processing from synchronous request handling.

6.3 Design Principles

Separation of Concerns: Each of the eight service modules owns a distinct domain (Section 7); no module directly accesses another module's database tables.

Single Responsibility: Each component within a module (Section 8) is responsible for one cohesive function (e.g., a dedicated EscalationEngine distinct from NotificationDispatcher).

Security by Design: Authentication, authorization, and encryption are enforced at the API gateway and data layer by default, not layered on afterward (Section 17).

Least Privilege: The permission model (FR-A4) is enforced server-side on every request, not only in client UI logic.

Fail-Safe Defaults: Safety-critical paths (Panic Mode, FR-D2) bypass optional gating rather than fail silently if a permission check is ambiguous.

Testability and Maintainability: MVVM on the client and a layered service structure on the backend keep business logic independent of UI and transport concerns, directly supporting NFR-11.

7. Module Design

7.1 Module Responsibilities

Module	Responsibility	Related SRS Requirements
Identity & Access Module	Registration, KYC/liveness verification, Guardian identity verification, permission dashboard, Guardian-Dependent linkage	FR-A1–A4, FR-B1–B4
Location & Geofence Module	Continuous location tracking, geofence definition and breach detection, location history, device battery and connectivity monitoring	FR-C1–C4, FR-L1, FR-L2
Emergency & Escalation Module	Panic Mode, Remote Alarm Override, Guardian Wellness Check, multi-level escalation, external emergency contact, fall detection	FR-D1–D6, FR-R1
Communication Module	Text messaging, voice/video calling, priority voice messages, multi-guardian bridge calls	FR-E1, E2, E4, E5
Medication & Records Module	Medication scheduling and confirmation, medical vault, PDF export, appointment tracking, daily vitals, wearable data ingestion	FR-F1, FR-G1–G4, FR-H1–H2
Directory & Marketplace Module	Healthcare service directory, home-care provider directory and booking requests, shared activity log	FR-I1, FR-J1, FR-I2/J2
Security & Surveillance Module	On-demand camera viewing, event-triggered clip capture, encryption enforcement, audit logging, data retention	FR-K1, K2, FR-N1–N3, FR-O1
Notification Module	Priority-tiered push notification delivery and acknowledgment tracking, used by every other module	FR-M1, FR-M2

7.2 Component Relationships

Modules communicate through well-defined internal service interfaces rather than direct database access across module boundaries. The Emergency & Escalation Module depends on the Location & Geofence Module (for activity cross-checks, FR-D4.3) and the Medication & Records Module (for missed-dose detection, FR-D1); it depends on the Notification Module for alert delivery and acknowledgment tracking, and on the Communication Module's gateway interface for the External Emergency Contact Fallback. The Security & Surveillance Module provides encryption and audit-logging services consumed by the Medication

& Records and Identity & Access Modules, rather than each module implementing its own encryption logic. This dependency structure mirrors the requirement-level dependencies already documented throughout SRS Section 4 and is preserved unchanged in this design.

8. Component Design

Each module (Section 7) is decomposed into cohesive components. Component names below correspond directly to backend service classes referenced in the Traceability Matrix (Section 23).

Identity & Access Module

- AuthController — registration, login, token issuance
- KYCVerifier — CNIC/liveness verification integration
- PermissionManager — module-level grant/revoke enforcement
- LinkageService — Guardian-Dependent linkage and cap enforcement

Location & Geofence Module

- LocationIngestor — adaptive-frequency location updates
- GeofenceEvaluator — breach detection across multiple zones
- LocationHistoryService — retrieval of historical tracks
- DeviceHealthMonitor — battery-level and connectivity-loss detection (FR-L1, FR-L2)

Emergency & Escalation Module

- PanicModeHandler
- WellnessCheckScheduler
- EscalationEngine — multi-tier timeout and state management
- FallDetectionAnalyzer
- ExternalContactNotifier

Communication Module

- MessagingService
- CallSignalingService — RTC session setup
- PriorityVoiceMessageService

Medication & Records Module

- MedicationScheduler
- MedicalVaultService — record storage and PDF export
- VitalsService
- WearableSyncService — BLE GATT data ingestion
- AppointmentService

Directory & Marketplace Module

- HealthcareDirectoryService
- ProviderDirectoryService
- BookingRequestService
- ActivityLogService

Security & Surveillance Module

- CameraStreamService
- ClipCaptureService
- EncryptionService
- AuditLogService
- RetentionPolicyEnforcer

Notification Module

- NotificationDispatcher — priority-tiered delivery
- AcknowledgmentTracker

9. Database Design

Humsafar uses PostgreSQL as its primary relational data store, selected for its strong ACID guarantees and mature support for foreign-key and check constraints, both important given the permission- and consent-sensitive nature of the data (medical records, location history, escalation state).

9.1 Entity-Relationship Diagram

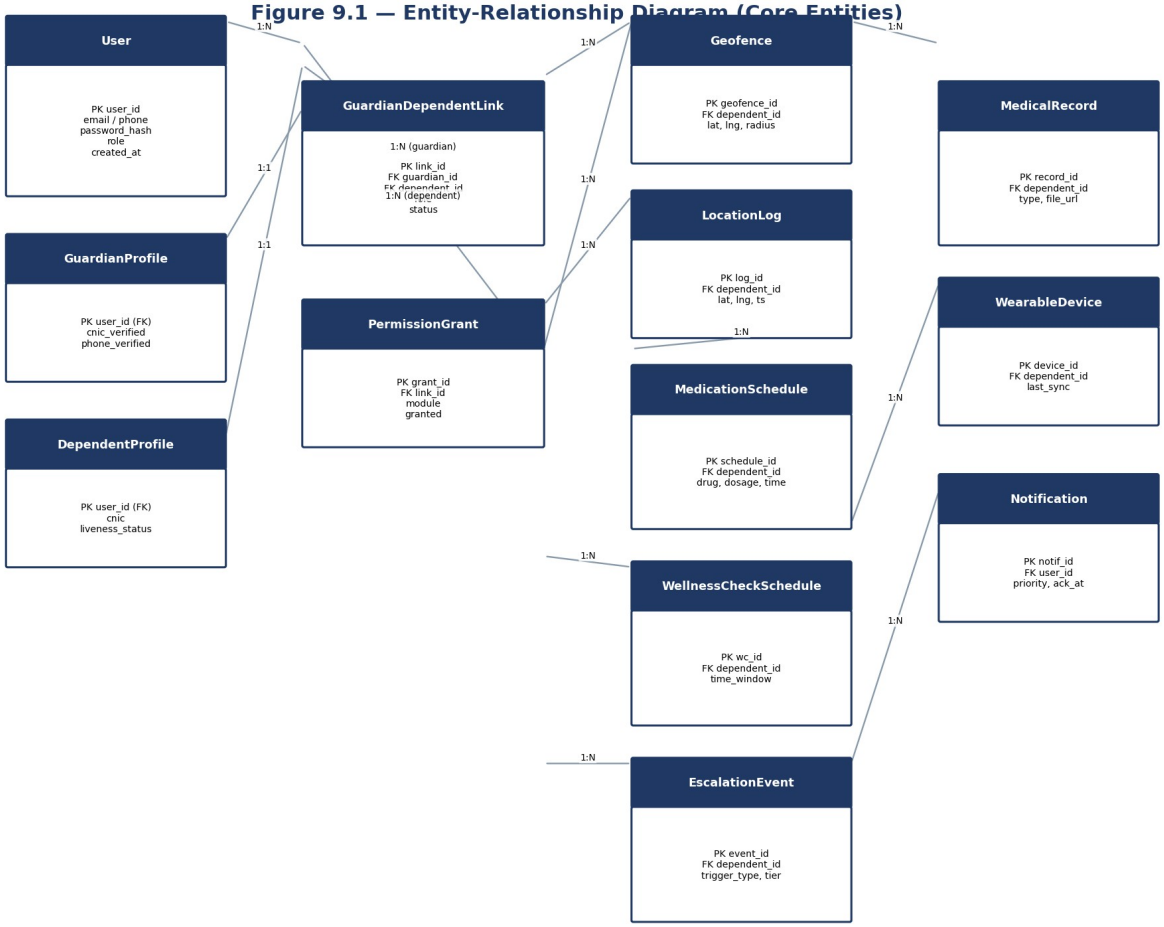


Figure 9.1 — Entity-Relationship Diagram (Core Entities)

The diagram above shows the twelve core entities central to the system's safety and access-control logic. The complete schema, including supporting tables for communication, directory, and audit data, is listed in Section 9.2.

9.2 Database Schema

Table	Key Columns	Traces To
users	user_id (PK), email, phone, password_hash, role, created_at	FR-A1
guardian_profiles	user_id (PK, FK→users), cnic_verified,	FR-A1, A3

	phone_verified, guardian_role	
dependent_profiles	user_id (PK, FK→users), cnic, liveness_status, dob, condition_notes	FR-A2
guardian_dependent_links	link_id (PK), guardian_id (FK), dependent_id (FK), role, status, linked_at	FR-B1, B2, B4
permission_grants	grant_id (PK), link_id (FK), module, granted, updated_at	FR-A4
geofences	geofence_id (PK), dependent_id (FK), name, lat, lng, radius_m, created_by	FR-C2, C3
location_logs	log_id (PK), dependent_id (FK), lat, lng, recorded_at, battery_level	FR-C1, C4, L1
medication_schedules	schedule_id (PK), dependent_id (FK), drug_name, dosage, time_of_day, frequency, created_by	FR-F1
medication_confirmations	confirmation_id (PK), schedule_id (FK), scheduled_for, confirmed_at, status	FR-F1, D1
wellness_check_schedules	wc_id (PK), dependent_id (FK), time_window, grace_period_min	FR-D4
wellness_check_responses	response_id (PK), wc_id (FK), prompted_at, responded_at, status	FR-D4
escalation_events	event_id (PK), dependent_id (FK), trigger_type, tier, status, created_at, resolved_at	FR-D1, D4, D5
emergency_contacts	contact_id (PK), dependent_id (FK), name, phone	FR-R1
medical_records	record_id (PK), dependent_id (FK), type, file_url, uploaded_by, uploaded_at	FR-G1, G2, G3
vital_readings	reading_id (PK), dependent_id (FK), metric_type, value, recorded_by, recorded_at, source	FR-H1, H2
appointments	appointment_id (PK), dependent_id (FK), provider_name, scheduled_at, notes	FR-G4
wearable_devices	device_id (PK), dependent_id (FK), ble_address, paired_at, last_sync_at	FR-H2
healthcare_organizations	org_id (PK), name, type, phone, lat, lng	FR-I1
home_care_providers	provider_id (PK), name, specialization, service_charge, contact	FR-J1
booking_requests	request_id (PK), dependent_id (FK), provider_id (FK), status, requested_at	FR-J1
activity_log	activity_id (PK), dependent_id (FK), type, reference_id, occurred_at	FR-I2/J2
messages	message_id (PK), sender_id (FK), receiver_id	FR-E1, E4

	(FK), type, content_url, priority, sent_at	
call_logs	call_id (PK), caller_id (FK), callee_id (FK), type, started_at, duration_s	FR-E2, E5
notifications	notification_id (PK), user_id (FK), type, priority, payload, sent_at, acknowledged_at	FR-M1, M2
camera_devices	camera_id (PK), dependent_id (FK), stream_uri, paired_at	FR-K1
camera_clips	clip_id (PK), dependent_id (FK), trigger_type, file_url, captured_at	FR-K2
audit_logs	audit_id (PK), actor_id (FK), action, target_type, target_id, occurred_at	FR-N3

9.3 Relationships

- Each **dependent_profiles** row relates to one or more **guardian_dependent_links** rows (1:N), each referencing exactly one **guardian_profiles** row, enforcing the 1 Primary + up to 2 Secondary cap (FR-B2) via an application-level constraint checked on insert.
- Each **guardian_dependent_links** row owns zero or more **permission_grants** rows (1:N), one per governable module (FR-A4.3).
- Each **dependent_profiles** row owns its own **location_logs**, **medication_schedules**, **wellness_check_schedules**, **medical_records**, **vital_readings**, and **wearable_devices** (all 1:N), consistent with the Dependent being the anchor entity for nearly all safety and health data.
- **escalation_events** references **dependent_profiles** directly and is populated by triggers from both **medication_confirmations** (missed-dose path, FR-D1) and **wellness_check_responses** (FR-D4), reflecting the shared escalation chain (FR-D5) described in SRS Section 4.2.
- **activity_log** is the shared table implementing the FR-I2/J2 merge decided during requirements review, referencing either a **healthcare_organizations** contact event or a **booking_requests** row via a polymorphic `reference_id` and `type` discriminator.

9.4 Constraints

- **Foreign key cascade rules:** deleting a `dependent_profiles` row cascades to delete all dependent-owned rows (location, medication, records, etc.) only when executed via the account-closure workflow governed by the data retention policy (FR-O1); direct cascading deletes are otherwise disabled to prevent accidental data loss.
- **Unique constraints:** (`guardian_id`, `dependent_id`) is unique within `guardian_dependent_links` to prevent duplicate linkage; (`dependent_id`, `role='primary'`) is enforced unique via a partial index to guarantee exactly one Primary Guardian per Dependent (FR-B2.2).
- **Check constraints:** `guardian_dependent_links.role` is restricted to `('primary','secondary')`; `escalation_events.tier` is restricted to an ordered enumeration reflecting the FR-D5 escalation sequence.
- **Not-null constraints** are applied to all foreign keys and to fields gating safety-critical logic (e.g., `medication_confirmations.status`, `wellness_check_responses.status`).

10. User Interface Design

10.1 Screen Descriptions

Screen	Primary Actor(s)	Description
Login / Registration	Guardian & Dependent	Role selection, credential entry, KYC capture flow for Dependents (FR-A1, A2, A3)
Guardian Dashboard	Guardian	Overview of linked Dependent(s), quick status (battery, last activity, pending alerts); switch-Dependent control (FR-B4.2)
Permission Management	Primary Guardian	Module-level grant/revoke controls for each Secondary Guardian (FR-A4)
Live Map	Guardian	Interactive map with Dependent location, geofence overlays, history playback (FR-C1–C4)
Medical Vault	Guardian & Dependent	Record list, upload, PDF export options, appointment list (FR-G1–G4)
Analytics Dashboard	Guardian	Adherence and vitals trend charts, summary reports (FR-Q1, Q2)
Communication	Guardian & Dependent	Message thread, call initiation, priority voice message recorder (FR-E1, E2, E4)
Directory / Marketplace	Guardian & Dependent	Healthcare organization directory, home-care provider browse and booking request (FR-I1, J1)
Camera View	Guardian	On-demand live stream, event clip playback (FR-K1, K2)
Dependent Mode — Home	Dependent	Single-screen entry point to the three safety-critical actions, large-target layout (FR-P1, P2)
Dependent Mode — Panic	Dependent	One-tap emergency trigger (FR-D2)
Dependent Mode — Wellness Check	Dependent	Confirmation prompt shown at scheduled times (FR-D4)
Dependent Mode — Medication Confirmation	Dependent	Two-step confirmation control (FR-F1.3)

10.2 Navigation Flow

The Guardian application uses a bottom-navigation structure with five top-level destinations — Dashboard, Live Map, Communication, Medical Vault, and Analytics — with Permission Management and Directory/Marketplace accessible from the Dashboard. The Dependent application defaults to Dependent Mode's single-screen home, from which the three safety-critical actions are reachable within one tap (NFR-07); standard (non-safety-critical) screens such as record viewing and appointments are accessible through a secondary menu, consistent with the SRS Section 4.7 scoping of simplified navigation to only the three safety-critical screens.

11. API and Backend Design

11.1 Endpoints

Method	Endpoint	Description	Auth
POST	/auth/register	Register a new Guardian or Dependent account	None
POST	/auth/login	Authenticate and issue JWT access/refresh tokens	None
POST	/dependents/{id}/kyc-verify	Submit CNIC and liveness capture for verification	JWT
POST	/dependents/{id}/link	Link a Dependent to a Primary Guardian	JWT
POST	/dependents/{id}/guardians/invite	Invite a Secondary Guardian	JWT (Primary)
PUT	/dependents/{id}/permissions/{module}	Grant or revoke a Secondary Guardian's module access	JWT (Primary)
GET	/dependents/{id}/location	Retrieve current/historical location	JWT + permission check
POST	/dependents/{id}/geofences	Create or update a geofence zone	JWT (Primary)
POST	/dependents/{id}/medications	Create a medication schedule	JWT + permission check
POST	/medications/{scheduleId}/confirm	Confirm a dose taken	JWT (Dependent)
POST	/dependents/{id}/wellness-checks	Configure Wellness Check schedule(s)	JWT + permission check
POST	/wellness-checks/{id}/respond	Submit a Wellness Check response	JWT (Dependent)
POST	/dependents/{id}/panic	Trigger Panic Mode	JWT (Dependent)
POST	/dependents/{id}/alarm	Trigger Remote Alarm Override	JWT (Primary only)
GET	/dependents/{id}/camera/stream	Request an on-demand live camera stream	JWT + permission check
GET	/dependents/{id}/records	List/retrieve medical records	JWT + permission check
POST	/dependents/{id}/records/export	Generate a PDF export (complete/year/month)	JWT + permission check
POST	/dependents/{id}/vitals	Record a manual vitals reading	JWT + permission check
POST	/wearables/sync	Ingest synced wearable data (heart rate, steps)	JWT (Dependent device)
GET	/directory/healthcare	Browse the healthcare service directory	JWT

GET	/directory/providers	Browse the home-care provider directory	JWT
POST	/providers/{id}/booking-requests	Submit a booking request	JWT + permission check
POST	/messages	Send a text or priority voice message	JWT
POST	/calls	Initiate a voice/video call session	JWT
GET	/notifications	Retrieve notifications; acknowledge via PATCH	JWT
GET	/dependents/{id}/analytics	Retrieve adherence and vitals analytics	JWT + permission check
GET	/dependents/{id}/audit-log	Retrieve the sensitive-action audit log	JWT (Primary)

11.2 Request/Response Structures

Representative examples are provided below; the complete OpenAPI specification is maintained as a separate build artifact.

POST /dependents/{id}/panic — Request:

{ "triggeredAt": "ISO-8601 timestamp" }

POST /dependents/{id}/panic — Response (200):

{ "eventId": "uuid", "status": "NOTIFIED", "guardiansNotified": ["guardianId", ...] }

POST /dependents/{id}/wellness-checks — Request:

{ "timeWindows": ["08:00", "18:00"], "gracePeriodMinutes": 30 }

Standard error envelope (all endpoints):

{ "error": { "code": "string", "message": "string", "details": {} } }

11.3 Authentication

Authentication uses stateless **JWT access tokens** (short-lived, approximately 15 minutes) paired with longer-lived refresh tokens, chosen for compatibility with the load-balanced application-server cluster (Section 16) without requiring shared session storage. Guardian identity verification (FR-A3) uses a phone-based OTP flow prior to first token issuance. Dependent KYC (FR-A2) is handled through a dedicated verification endpoint that forwards CNIC and liveness data to the third-party KYC provider (Section 3.3) and does not itself issue authentication tokens until verification succeeds. All endpoints enforce permission checks server-side against the `permission_grants` table (Section 9.2), independent of any client-side UI restriction, consistent with the least-privilege principle (Section 6.3).

12. Sequence Diagrams

The following diagrams illustrate the three most safety-critical interaction flows in the system. Each message is annotated with the functional requirement it implements.

Figure 12.1 — Sequence Diagram: Guardian-Dependent Registration and Linkage

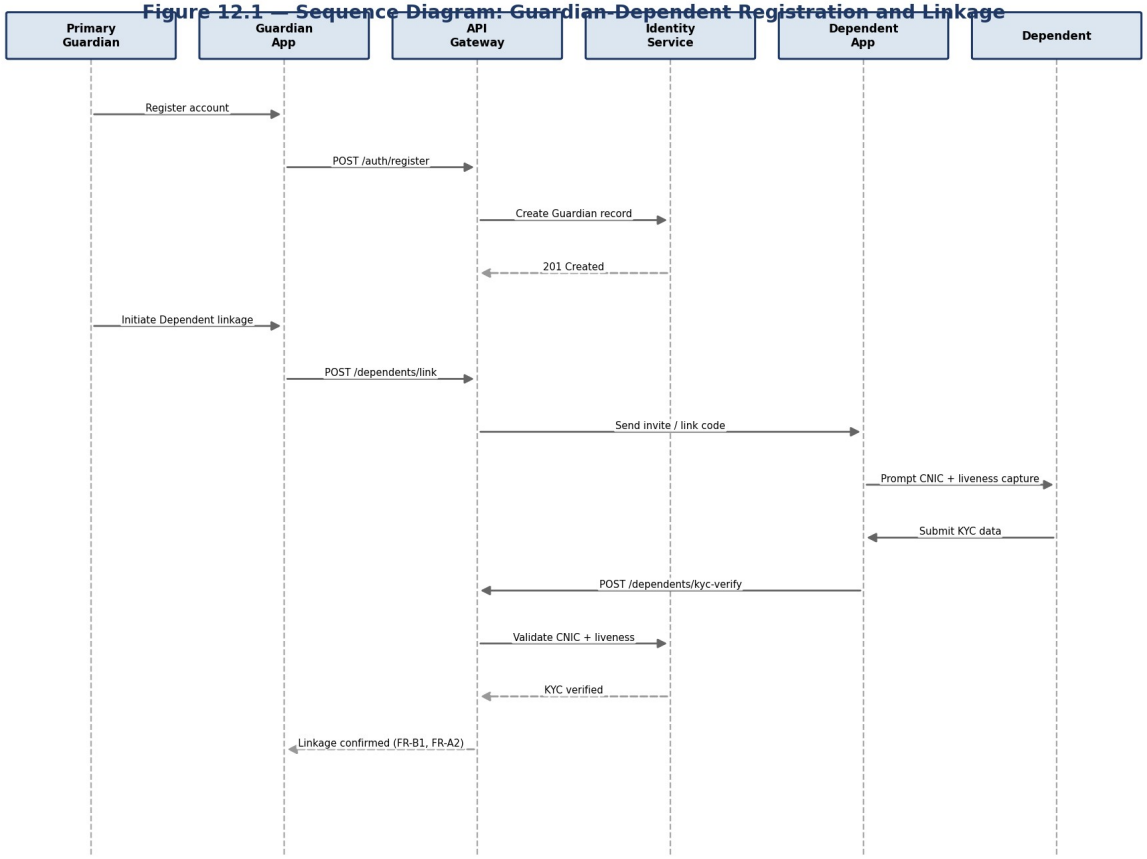


Figure 12.1 — Guardian-Dependent Registration and Linkage

This flow implements FR-A1 through FR-A3 and FR-B1: a Primary Guardian registers, initiates linkage with a Dependent, and the Dependent completes CNIC and liveness verification before the account activates.

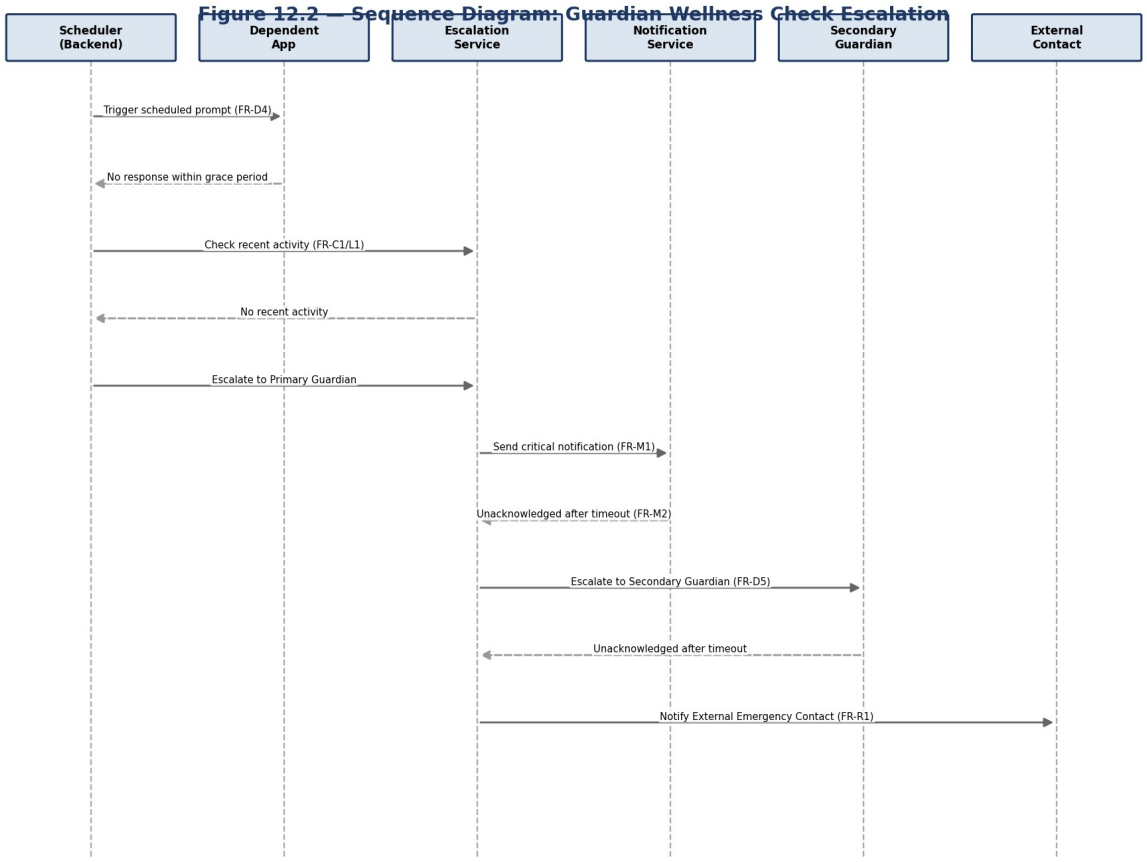


Figure 12.2 — Guardian Wellness Check Escalation

This flow implements FR-D4 and FR-D5: a missed Wellness Check response, cross-checked against recent device activity, escalates through Primary then Secondary Guardians before reaching the External Emergency Contact (FR-R1).

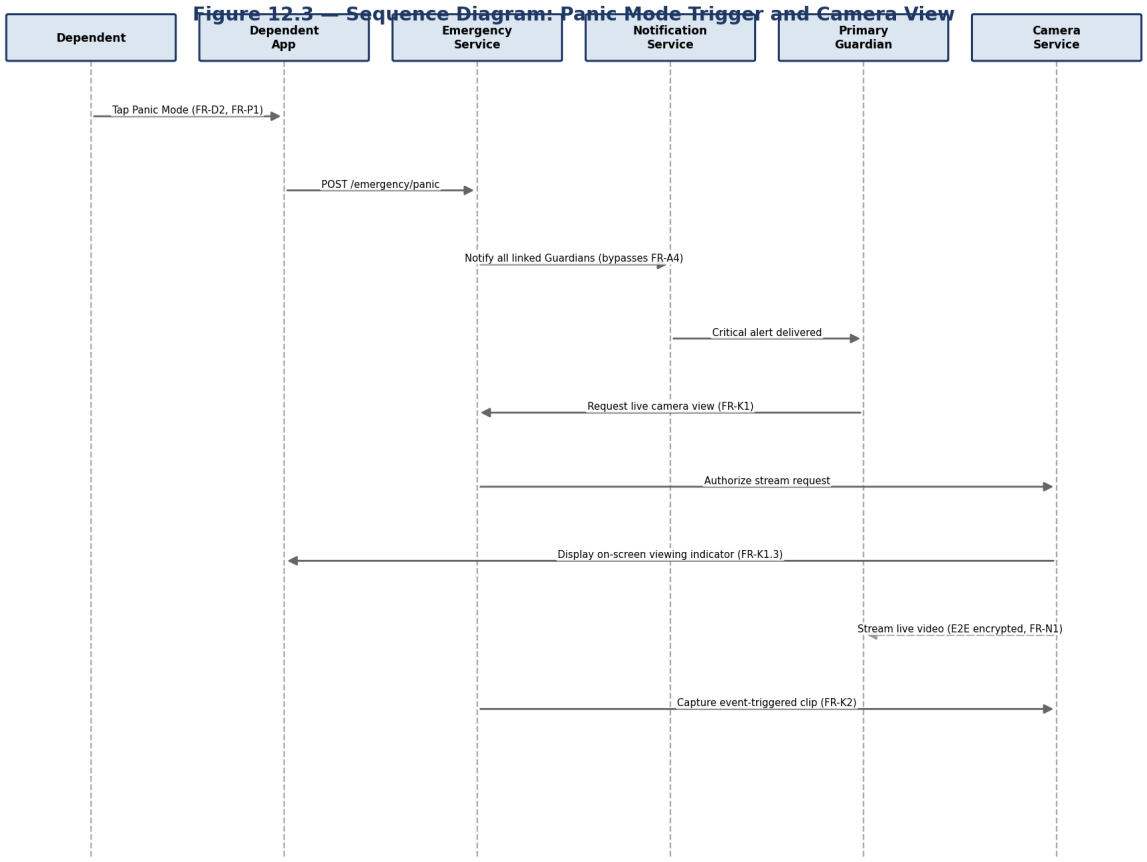


Figure 12.3 — Panic Mode Trigger and Camera View

This flow implements FR-D2, FR-K1, and FR-N1: Panic Mode bypasses standard permission gating to notify all Guardians immediately, after which a Guardian may request a live, encrypted camera view, triggering the on-screen viewing indicator required by FR-K1.3.

13. Activity Diagrams

Figure 13.1 — Activity Diagram: Medication Confirmation and Escalation

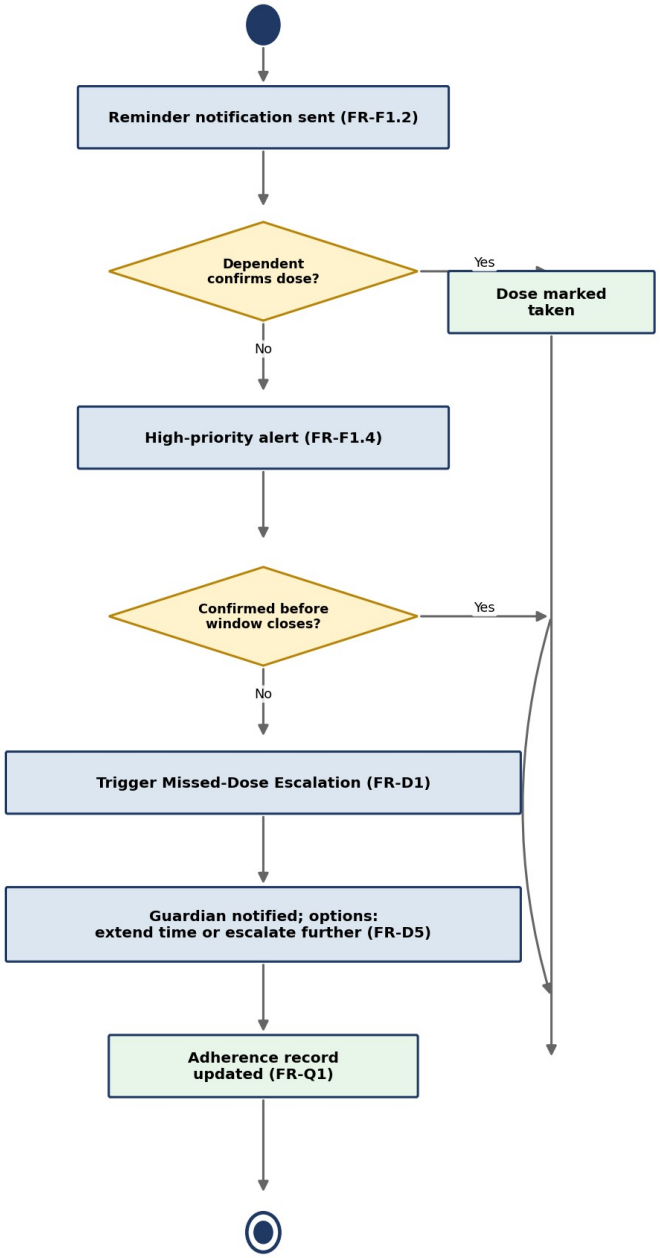


Figure 13.1 — Medication Confirmation and Escalation

Implements FR-F1 and FR-D1: reminders, dual confirmation, and automatic escalation on a missed, unconfirmed dose, feeding the adherence analytics dashboard (FR-Q1).

Figure 13.2 — Activity Diagram: Emergency Detection and Response

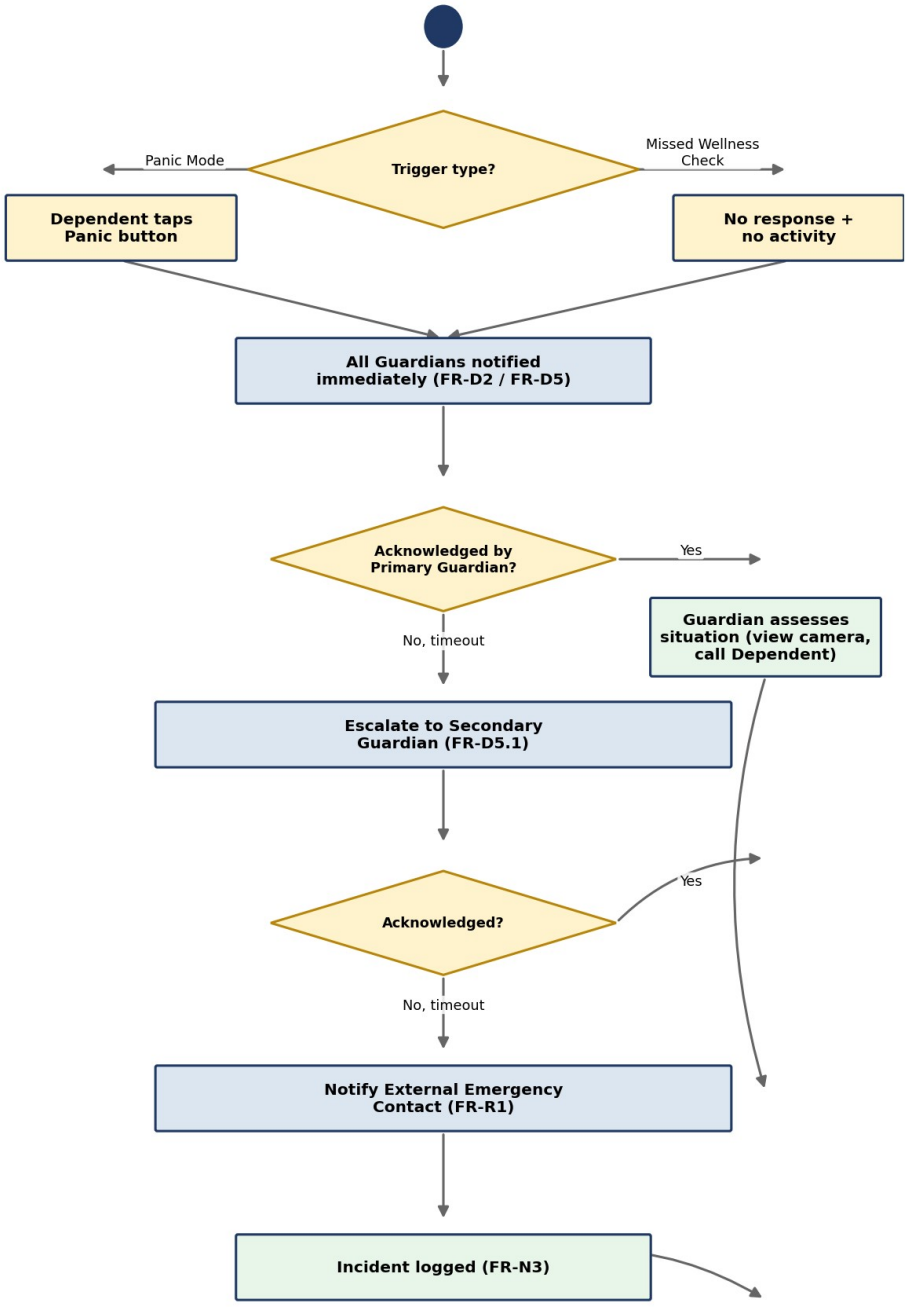


Figure 13.2 — Emergency Detection and Response

Implements the combined Panic Mode (FR-D2) and Wellness Check escalation (FR-D5) paths, converging on a single notification and audit-logging flow (FR-N3) regardless of which mechanism detected the emergency.

14. Class Diagram

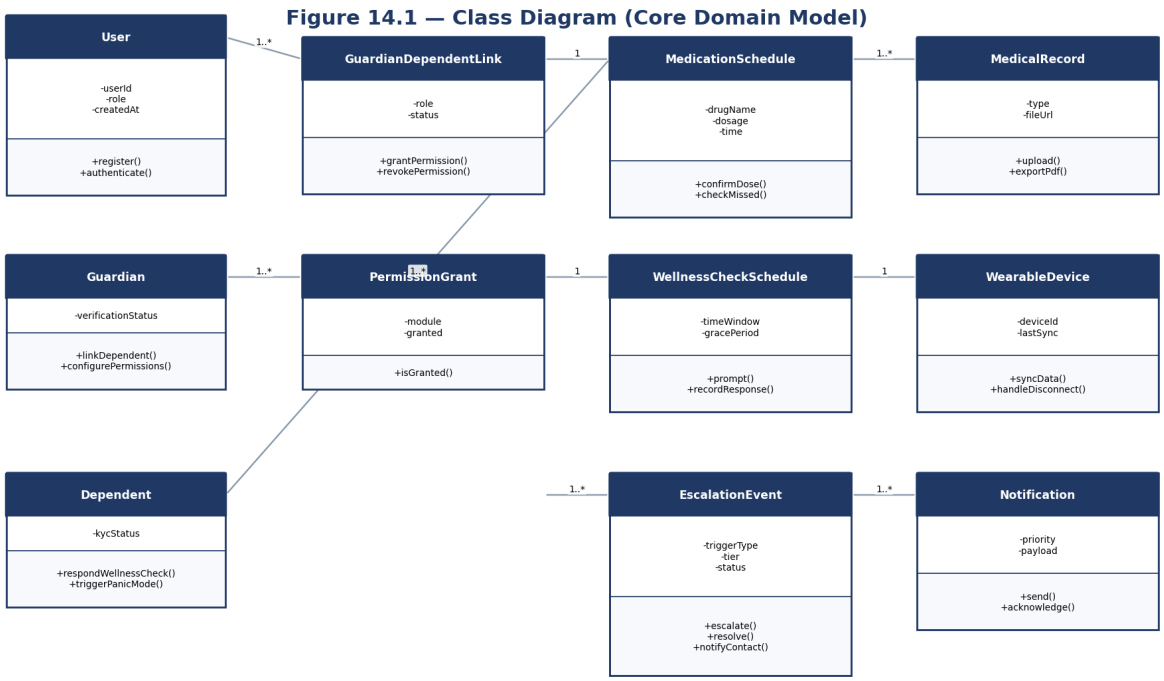


Figure 14.1 — Core Domain Model

The class diagram reflects the same core entities as the ER diagram (Section 9.1), expressed as domain classes with their primary behaviors. `Guardian` and `Dependent` specialize the base `User` class; `EscalationEvent` is the class most central to the system's safety guarantees, since both `MedicationSchedule` and `WellnessCheckSchedule` raise escalations through it.

15. State Diagrams

Figure 15.1 — State Diagram: Escalation Alert Lifecycle

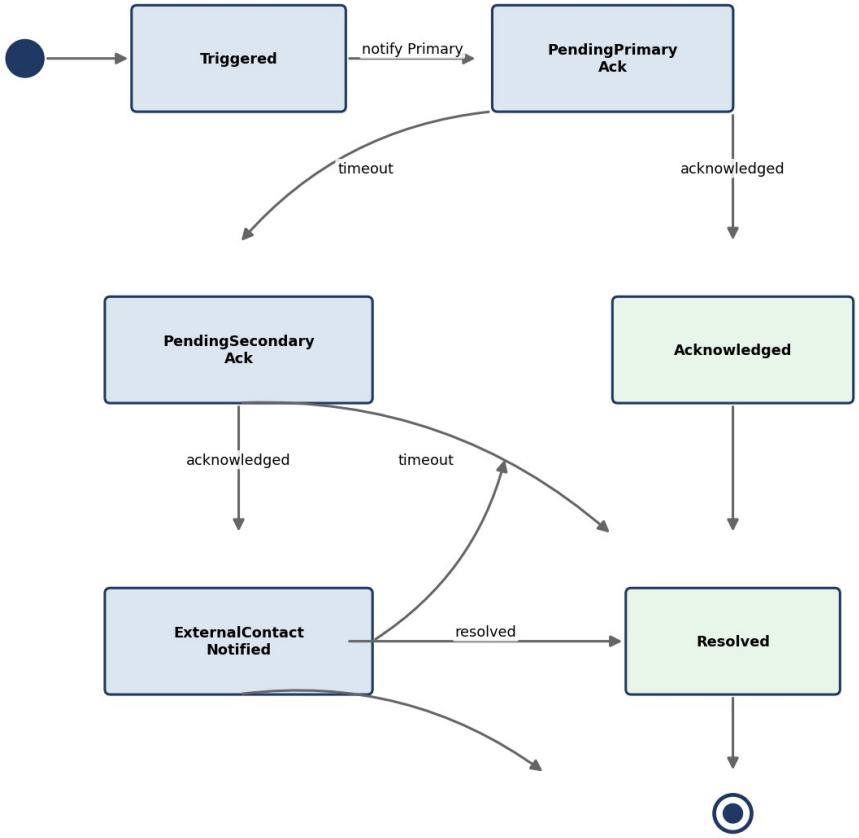


Figure 15.1 — Escalation Alert Lifecycle

Models the state transitions of an `EscalationEvent` (FR-D1, D4, D5, R1) from creation through resolution, including the timeout-driven transitions between Primary Guardian, Secondary Guardian, and External Emergency Contact notification tiers.

Figure 15.2 — State Diagram: Dependent Account Activation

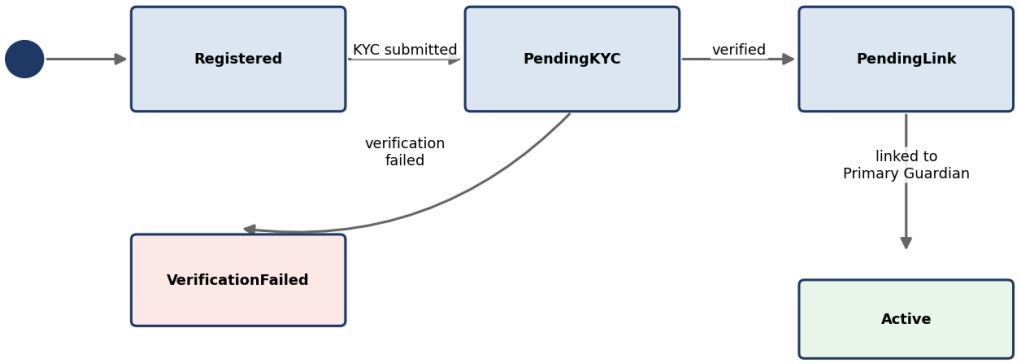


Figure 15.2 — Dependent Account Activation

Models the Dependent account lifecycle from registration through KYC verification (FR-A2) to activation, which requires both successful verification and linkage to a Primary Guardian (FR-B1.3).

16. Deployment Architecture

Figure 16.1 — Deployment Architecture

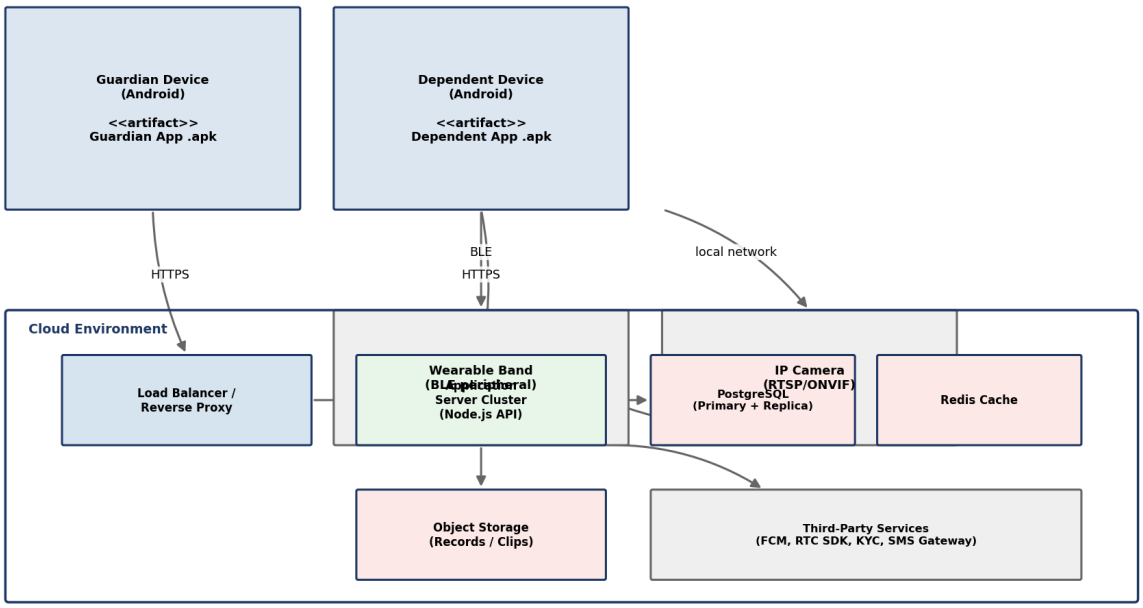


Figure 16.1 — Deployment Architecture

The Guardian and Dependent Android applications communicate with a cloud-hosted application-server cluster over HTTPS, fronted by a load balancer to satisfy the concurrent-connection requirement of NFR-06. The application-server cluster is stateless (session state lives in JWTs and Redis), allowing horizontal scaling. PostgreSQL runs as a primary instance with a read replica to support analytics queries (FR-Q1) without contending with transactional writes from safety-critical paths. The wearable band and IP camera connect locally to the Dependent device over BLE and the local network respectively, consistent with SRS Section 3.2's hardware interface requirements.

17. Security Design

Authentication & Authorization: JWT-based stateless authentication (Section 11.3); server-side role and permission enforcement on every request against `permission_grants` (FR-A4), independent of client logic.

Identity Verification: CNIC and liveness verification for Dependents (FR-A2) and lighter-weight CNIC/OTP verification for Guardians (FR-A3), both via a third-party KYC provider rather than in-house biometric matching.

Encryption in Transit: All client-server traffic over TLS 1.2+; all real-time audio/video streams end-to-end encrypted (FR-N1, NFR-03).

Encryption at Rest: Medical records, health metrics, and location history encrypted at the database/storage layer (FR-N2, NFR-04).

Audit Logging: All permission changes, camera-view events, and medical-record access events are recorded in `audit_logs` (FR-N3), viewable by the Primary Guardian.

Camera Access Control: Camera streaming gated by permission dashboard; the Dependent's device displays an on-screen indicator during active viewing (FR-K1.3), providing notice rather than silent monitoring.

BLE Pairing Security: Wearable band pairing uses standard BLE bonding; the application does not accept GATT data from an unpaired or unexpected device address.

Input Validation & API Hardening: All API inputs validated server-side against a defined schema before processing; parameterized queries used throughout to prevent injection; rate limiting applied to authentication and KYC endpoints.

Data Retention: Automated enforcement of the retention/deletion policy (FR-O1) via a scheduled job, rather than manual cleanup.

18. Error Handling and Exception Strategy

All API responses use the standard error envelope defined in Section 11.2, carrying a machine-readable `code`, a human-readable `message`, and a `details` object for field-level validation errors. Client applications map error codes to localized, Dependent Mode-appropriate messaging (FR-P2, FR-P4) rather than surfacing raw backend errors.

- **Network/BLE transient failures:** the Wearable Sync Service retries with exponential backoff and surfaces a soft "last synced at" indicator rather than a hard failure (FR-H2.4).
- **Third-party service failures (KYC, RTC, push, SMS gateway):** requests are queued and retried; a persistent KYC provider outage surfaces as a "verification pending" state rather than blocking registration indefinitely.
- **Escalation delivery failures:** if a push notification fails to send, the Escalation Engine treats this identically to a non-acknowledgment for timeout purposes, rather than silently stalling the escalation chain (FR-D5).
- **Validation errors:** rejected at the API boundary with field-level detail before reaching business logic, preventing malformed data (e.g., an invalid geofence radius) from being persisted.
- **Backend unavailability:** per NFR-05, the mobile applications queue outgoing critical alerts locally and retry on reconnection rather than discarding them.

19. Logging and Monitoring Strategy

Two distinct logging mechanisms serve different purposes and are not conflated: **application logs** (structured JSON, including a correlation ID per request) support debugging and operational monitoring, while **audit logs** (Section 9.2, `audit_logs`) are a security and accountability record of sensitive actions (FR-N3) and are retained under the data-retention policy (FR-O1) rather than rotated on a short operational cycle.

- **Monitored metrics:** API latency (against NFR-01's 2-second target), backend uptime (against NFR-05's 99.9% target), escalation-chain completion time, and BLE sync success rate (against NFR-13).
- **Alerting:** automated alerts fire on sustained latency degradation, elevated error rates, or any failure in the escalation-delivery path, given its direct safety relevance.
- **Correlation:** every request carries a correlation ID propagated through logs across the API gateway and all invoked service modules, allowing a single incident (e.g., a delayed Panic Mode notification) to be traced end-to-end.

20. Performance and Scalability Considerations

- **Adaptive location polling** (FR-C1) is the primary lever for satisfying NFR-02's battery budget, reducing update frequency while the Dependent is stationary and increasing it during movement.
- **Caching:** directory listings (FR-I1, FR-J1) and analytics aggregates (FR-Q1) are cached in Redis with short TTLs, reducing repeated load on PostgreSQL for read-heavy, low-volatility data.
- **Horizontal scaling:** the stateless application-server cluster (Section 16) scales horizontally behind the load balancer to satisfy NFR-06's concurrent-Guardian requirement without architectural change.
- **Database indexing:** indexes on `dependent_id` across all Dependent-owned tables, and a partial unique index enforcing the single-Primary-Guardian constraint (Section 9.4), keep the most frequent query patterns efficient.
- **Command latency:** the Escalation Engine and Notification Dispatcher run as high-priority queue consumers, isolated from lower-priority background jobs (e.g., analytics aggregation), to protect NFR-01's 2-second command-latency target under load.

21. Design Constraints

- **Android-only target** (SRS 2.4) removes any need for a cross-platform abstraction layer; the client codebase is written natively in Kotlin without a compatibility shim for a second platform.
- **No offline mode** (SRS 2.5) allows the design to assume network connectivity as the default case; synchronization logic does not need to reconcile extended offline edit histories.
- **Three-person development team, single academic-year timeline** directly motivates the modular-monolith architectural choice (Section 6.2) over microservices.
- **Wearable integration via standard BLE GATT only** (FR-H2) constrains the Wearable Sync Service to the Heart Rate and Battery GATT services; no vendor-specific SDK integration is designed.
- **No formal HIPAA/GDPR certification claimed** (SRS 2.5) means the security design (Section 17) follows general data-protection best practice rather than a specific regulatory compliance checklist.

22. Design Decisions and Rationale

Decision	Alternative(s) Considered	Rationale
Modular monolith over microservices	Independent, per-service deployment pipelines and distributed-transaction handling	Matches a three-person team and single-year timeline; preserves clean module boundaries for a possible future split
PostgreSQL over a document/NoSQL store	MongoDB-style document store	Data is highly relational (permissions, escalation chains, guardian caps); foreign-key and check constraints enforce business rules the database itself, reducing application-level bugs
REST over GraphQL	GraphQL API layer	Simpler tooling and team familiarity within the project timeline; the API's resource shape is stable and well-suited to REST
JWT (stateless) over server-side sessions	Session store shared across app-server instances	Enables horizontal scaling behind a load balancer (NFR-06) without a shared session backend
Kotlin + MVVM over a cross-platform framework	Flutter / React Native	Android-only scope (SRS 2.4) removes the main benefit of cross-platform tooling; native Kotlin gives full access to BLE and background-service APIs needed for FR-C1/H2
Redis for both cache and lightweight queueing	Separate dedicated message broker (e.g., RabbitMQ)	Reduces the number of infrastructure components a three-person team must operate, at a scale where a dedicated broker is not yet justified
On-screen camera-viewing indicator (FR-K1.3)	No indicator (silent monitoring)	Addresses the privacy/consent concern raised during requirements review (SRS Section 4.6) at negligible engineering cost

23. Requirements Traceability Matrix

Maps each Functional Requirement group from the SRS to the design module, component(s), and database table(s) that realize it, closing the loop between requirements and design.

FR ID	Design Module	Component(s)	Database Table(s)
FR-A1	Identity & Access	AuthController	users, guardian_profiles
FR-A2	Identity & Access	KYCVerifier	dependent_profiles
FR-A3	Identity & Access	AuthController, KYCVerifier	guardian_profiles
FR-A4	Identity & Access	PermissionManager	permission_grants
FR-B1	Identity & Access	LinkageService	guardian_dependent_links
FR-B2	Identity & Access	LinkageService	guardian_dependent_links (unique constraint)
FR-B4	Identity & Access	LinkageService	guardian_dependent_links
FR-C1	Location & Geofence	LocationIngestor	location_logs
FR-C2	Location & Geofence	GeofenceEvaluator	geofences
FR-C3	Location & Geofence	GeofenceEvaluator	geofences
FR-C4	Location & Geofence	LocationHistoryService	location_logs
FR-L1	Location & Geofence	DeviceHealthMonitor	location_logs (last-known-location path)
FR-L2	Location & Geofence	DeviceHealthMonitor	— (event-driven, no dedicated table)
FR-D1	Emergency & Escalation	EscalationEngine	medication_confirmations, escalation_events
FR-D2	Emergency & Escalation	PanicModeHandler	escalation_events
FR-D3	Emergency & Escalation	PanicModeHandler (alarm path)	—
FR-D4	Emergency & Escalation	WellnessCheckScheduler	wellness_check_schedules, wellness_check_responses
FR-D5	Emergency & Escalation	EscalationEngine	escalation_events
FR-D6	Emergency & Escalation	FallDetectionAnalyzer	escalation_events
FR-R1	Emergency & Escalation	ExternalContactNotifier	emergency_contacts
FR-E1	Communication	MessagingService	messages
FR-E2	Communication	CallSignalingService	call_logs
FR-E4	Communication	PriorityVoiceMessageService	messages
FR-E5	Communication	CallSignalingService	call_logs
FR-F1	Medication & Records	MedicationScheduler	medication_schedules,

			medication_confirmations
FR-G1	Medication & Records	MedicalVaultService	medical_records
FR-G2	Medication & Records	MedicalVaultService	medical_records, permission_grants
FR-G3	Medication & Records	MedicalVaultService	medical_records
FR-G4	Medication & Records	AppointmentService	appointments
FR-H1	Medication & Records	VitalsService	vital_readings
FR-H2	Medication & Records	WearableSyncService	wearable_devices, vital_readings
FR-I1	Directory & Marketplace	HealthcareDirectoryService	healthcare_organizations
FR-J1	Directory & Marketplace	ProviderDirectoryService, BookingRequestService	home_care_providers, booking_requests
FR-I2/J2	Directory & Marketplace	ActivityLogService	activity_log
FR-K1	Security & Surveillance	CameraStreamService	camera_devices
FR-K2	Security & Surveillance	ClipCaptureService	camera_clips
FR-N1	Security & Surveillance	EncryptionService	— (transport-level)
FR-N2	Security & Surveillance	EncryptionService	medical_records, vital_readings, location_logs
FR-N3	Security & Surveillance	AuditLogService	audit_logs
FR-O1	Security & Surveillance	RetentionPolicyEnforcer	all Dependent-owned tables
FR-M1	Notification	NotificationDispatcher	notifications
FR-M2	Notification	AcknowledgmentTracker	notifications
FR-P1	Client (Dependent App)	Dependent Mode UI (Section 10)	— (client-side)
FR-P2	Client (Dependent App)	Dependent Mode UI (Section 10)	— (client-side)
FR-P4	Client (both apps)	Localization layer	— (client-side)
FR-Q1	Medication & Records	Analytics aggregation (Redis-cached)	medication_confirmations, vital_readings
FR-Q2	Medication & Records	MedicalVaultService (export path)	medical_records

24. Appendices

24.1 Technology Stack Summary

Component	Technology
Mobile Client	Kotlin, MVVM architecture, native Android (API 26–latest)
Backend API	Node.js with TypeScript, REST over HTTPS
Primary Database	PostgreSQL (primary + read replica)
Cache / Queue	Redis
Object Storage	Cloud object storage (medical records, camera clips)
Push Notifications	Firebase Cloud Messaging (FCM)
Real-Time Communication	Third-party RTC SDK (voice/video calling)
Identity Verification	Third-party KYC/liveness verification provider
SMS/Voice Gateway	Third-party telephony gateway (External Emergency Contact Fallback)
Wearable Integration	Bluetooth Low Energy, standard GATT profiles (Heart Rate 0x180D, Battery 0x180F)
Authentication	JWT (access + refresh tokens)

24.2 Glossary

All terminology follows SRS Section 7.1 without modification. Design-specific acronyms introduced in this document are defined in Section 4.3.